

Rendszerarchitektúra, Célkitűzések és Adatmodell

EV Charging Platform & Management System — Átfogó Rendszerterv (1. Rész)

1. A Projekt Célkitűzése és Piaci Pozícionálása

A jelenlegi európai és hazai elektromos jármű (EV) töltési piac komoly strukturális problémákkal küzd. A felhasználók nap mint nap szembesülnek a roaming hálózatok átláthatatlan káoszával, a töltés közben váratlanul megszakadó kommunikációval (ún. „ghost sessions”), amelyek során a rendszer semmit sem tölt, de a számláló fut, valamint a banki fizetéseknél tapasztalható megmagyarázhatatlanul beragadt zárolásokkal (tipikusan a 10 000 Ft-os „pre-auth” összegek, amelyek napokig blokkolják a felhasználó likviditását).

Az **EV Charging Platform** célja egy olyan robusztus, vállalati szintű, OCPP 1.6-J / 2.0.1 kompatibilis backend platform megvalósítása, amely teljesen kiküszöböli ezeket a hibákat. A rendszer alapvető filozófiája a *Zero-Risk Tranzakciókezelés* és az autonóm öngyógyítás (*self-healing*), garantálva a tökéletes transzparenciát és megbízhatóságot.

2. Rendszerarchitektúra & Technológiai Stack

A platform modern, mikroszolgáltatás-kompatibilis, de robusztus monolitikus Spring Boot architektúrára épül, amely könnyen skálázható felhő környezetbe (Docker / Kubernetes).

- **Backend Alapok:** Java 17 / 21, Spring Boot 3.2.4 (Spring Web, Spring WebSocket, Spring Data JPA).
- **Adatbázis Réteg:** PostgreSQL relációs adatbázis, amely biztosít az ACID tulajdonságok révén feltétlen integritást a pénzügyi és töltési tranzakciókhoz.
- **Protokoll Réteg:** OCPP 1.6-J (Open Charge Point Protocol - JSON over WebSockets), amely szabványosított, kétirányú valós idejű kommunikációt teremt a töltőoszlopok és a szerver között.

Adatbázis Séma és Zero-Risk Üzleti Logika

EV Charging Platform & Management System — Átfogó Rendszerterv (2. Rész)

3. Adatbázis Séma és Főbb Táblák

Az adatbázis megtervezésekor a legfőbb szempont a gyors elérhetőség és a tranzakciós naplózhatóság volt. A két fő alaptábla:

Tábla Neve	Főbb Mezők és Kulcsattribútumok	Leírás
charge_points	id, vendor, model, status (OFFLINE, AVAILABLE, CHARGING, FAULTED)	A hálózatra csatlakoztatott fizikai töltőoszlopok nyilvántartása és aktuális állapotuk.
charging_sessions	id, charge_point_id, start_time, end_time, kwh_consumed, cost_huf, status (PENDING, ACTIVE, COMPLETED, FAILED)	Aktuális és történeti töltési munkamenetek részletes adatai a pontos elszámoláshoz.

A Kritikus Hibákat Kiküszöbölő Üzleti Logika (Zero-Risk Model)

- 1. Atomic State Check:** A rendszer semmilyen körülmények között nem kezdeményez banki pénz zárolást (pre-auth), amíg a fizikai töltőoszlop OCPP WebSocketen keresztül vissza nem igazolja, hogy a csatlakozás stabil, a kábel rögzítve van, és a töltő készen áll az energiaközlésre.
- 2. Automatic Refund Scheduler:** Ha a tranzakció indítását követően a töltés 30 másodpercen belül nem vált át aktív (ACTIVE) státuszba (pl. kommunikációs timeout vagy hardverhiba miatt), a Spring Boot háttérfolyamat automatikus parancsot küld a banki gateway-nek a zárolás azonnali feloldására (Void), így megszüntetve a beragadt összegeket.

4. Következő Lépések és Fejlesztési Ütemterv

A rendszerterv birtokában a 4 fős csapat megkezdheti a munkát a megosztott GitHub repository dev ágán. Az 1. feladat a környezet kiépítése, a 2. feladat a WebSocket szerver felhúzása, a 3. feladat az állapotgép implementálása, míg a 4. feladat a fizetési integráció lezárása.